

EVALUACIÓN	Obligatorio 1	GRUPO	Todos	FECHA	
MATERIA	Diseño de aplicación 1				
CARRERA	AN e ID				
CONDICIONES	- Puntaje máximo: 20 puntos - Puntaje mínimo: 0 puntos - Fecha de entrega: 14/05/26 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - El uso de IAG está autorizado, corresponde al docente del curso establecer las pautas para su utilización. - Todo contenido producido por la IAG que forme parte de una entrega debe ser correctamente referenciado. - Se debe indicar: - la /las herramientas utilizadas y - el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc. - Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante. - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema. - La defensa será a aquellos grupos que corresponda, pudiendo ser todos los grupos o solamente la mitad, dependiendo de cada dictado. - La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: entre 15/05/2026 y 04/06/2026 El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. <u>La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.</u> IMPORTANTE: 1) Inscribirse 2) Formar grupos de hasta 3 personas del mismo dictado 3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: "RECORDATORIO") Aquellos de ustedes que <u>presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos</u>, por favor contactarse con el Coordinador de cursos o Coordinación adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.				

Obligatorio I de Diseño de Aplicaciones I

Proyecto WorldCupPlanner

Descripción General del Proyecto

WorldCupPlanner 2026 es una aplicación para gestionar la fase de grupos del Mundial de Fútbol (12 grupos de 4 equipos) y preparar los cruces mediante sorteos determinísticos (semillas), asegurando trazabilidad y reproducibilidad.

Contexto

El Mundial de Fútbol 2026 será el primero con 48 equipos y un formato renovado que plantea desafíos interesantes en la organización del torneo. El proyecto WorldCupPlanner 2026 busca simular este escenario mediante el desarrollo de una aplicación que permita gestionar la fase de grupos, generar el fixture de manera automática y organizar los cruces eliminatorios con reglas claras y sorteos reproducibles.

El objetivo es aplicar principios de diseño de software, arquitectura por capas, desarrollo guiado por pruebas (TDD) y buenas prácticas de programación, integrando funcionalidades que combinan lógica de negocio, algoritmos determinísticos y una interfaz amigable para el usuario.

Esta primera entrega se centra en la construcción de la base del sistema, asegurando la correcta gestión de entidades, la generación del fixture y la preparación para las fases eliminatorias, todo con trazabilidad y auditoría completa.

Requerimientos de Implementación

1. Requerimientos No Funcionales

1.1. Separación de Componentes

- La lógica de negocio y la interfaz de usuario deben estar claramente separadas, permitiendo una evolución independiente de cada uno.

1.2. Tecnología

- La aplicación debe desarrollarse utilizando C# en .NET Core 8 con Blazor Server App.
- El código fuente debe gestionarse mediante un repositorio en GitHub utilizando la metodología "gitflow".

1.3. Desarrollo Guiado por Pruebas (TDD)

- Todas las clases (sin tener en cuenta UI) deben ser diseñadas y construidas utilizando la metodología TDD (Test-Driven Development) usando [MSTest](#).
- Se debe asegurar que todas las reglas de negocio especificadas estén completamente implementadas y probadas.
- Para automatizar la ejecución de pruebas, se configurará un pipeline de GitHub Actions que ejecute los tests cada vez que se realice un pull request hacia la rama principal (main).
- Si bien no será obligatorio aprobar las pruebas para concretar el merge, el pipeline deberá ejecutarse siempre y reportar los resultados de manera visible, tal como fue presentado en la clase de Tecnologías.

2. Requerimientos Funcionales

2.1. Gestión de Usuarios (ABM)

- Roles (no excluyentes):
 - Administrador del Sistema: Gestión de usuarios; gestión de equipos; gestión de estadios; gestión y generación de equipos, visualizar logs del sistema.
 - Editor: Generar y consultar el fixture; editar partidos solo en atributos permitidos (fecha, estadio, resultado); simular resultados; realizar sorteos de cruces (Segunda fase); Importación.

Los roles no son excluyentes. Esto significa que un usuario puede desempeñar simultáneamente un rol general (por ejemplo, como Administrador del Sistema) y, además, tener roles específicos (como Editor).

- Información del Usuario: Nombre, apellido, email, fecha de nacimiento (MM-DD-AAAA) y contraseña.
- Contraseña: Los administradores pueden generar una contraseña por defecto o reiniciar la contraseña de un usuario existente. Los usuarios pueden definir su propia contraseña siempre y cuando estén logueados.

Requisitos de la Contraseña:

Longitud mínima: La contraseña debe tener al menos 8 caracteres.

Persistida con algún tipo de cifrado, no texto plano.

Combinación de caracteres:

- Debe incluir al menos una letra mayúscula (A-Z).
- Debe incluir al menos una letra minúscula (a-z).
- Debe incluir al menos un número (0-9).
- Debe incluir al menos un carácter especial (como @, #, \$, .).

2.2. Gestión de Equipos (CRUD)

Se tiene que soportar la creación, modificación y eliminación de un equipo.

Entidad Equipo (con validaciones y longitudes máximas):

- Nombre (obligatorio, 1–60 chars).
- Confederación (obligatorio; AFC, CAF, CONCACAF, CONMEBOL, OFC, UEFA).
- Ranking FIFA (obligatorio; entero 300–2500).

Reglas de carga/edición:

- Nombre único en el sistema.
- Cupos por confederación (simplificado): UEFA 16; CONMEBOL 7; CONCACAF 7; CAF 9; AFC 8; OFC 1.
- Empates de RankingFIFA se resuelven por sorteo determinístico (Fisher–Yates) usando SemillaFixture, auditando el orden resultante. Los equipos se deben ordenar del 1 al 48 para poder hacer el sorteo del fixture.

2.3. Gestión de Estadios (CRUD)

Se tiene que soportar la creación, modificación y eliminación de un estadio.

Entidad Estadio (con validaciones y longitudes máximas):

- Nombre (obligatorio, 1–80 chars).
- Ciudad (obligatorio, 1–60 chars).
- Descripción (opcional, 1–400 chars).
- Capacidad locativa - público (obligatorio).

Reglas de carga/edición:

- Nombre único en el sistema.
- Capacidad debe ser mayor o igual a 20000 espectadores

2.4. Generación automática de equipos

- Si hay menos de 48 equipos, permitir completar automáticamente hasta el total, respetando los cupos.
- Generar equipos ficticios con nombres determinísticos (UEFA_01, CONMEBOL_02, ...) y RankingFIFA reproducible (SemillaCompletar).
- Auditar cantidad completada por confederación, semilla usada y rangos asignados.

2.5. Edición de partido

Se debe poder editar la fecha o el estadio de un partido, o cargar el resultado del mismo

- Id (numérico incremental autogenerado).
- Fecha (obligatorio; formato ISO 8601; fecha válida).
- Estadio (obligatorio).
- Equipo local (obligatorio).
- Equipo visitante (obligatorio; distinto de Local).
- Grupo (obligatorio: etiqueta A–L).
- Goles local
- Goles visitante
- Vencedor

Una vez generados los cruces para la segunda fase del torneo (ver 2.8) no se deben poder editar los partidos de la fase anterior.

2.6 Simulación de Resultados

Se debe incluir una funcionalidad para simular el resultado de partidos de la fase de grupos y todas las restantes fases del torneo.

Algoritmo de Simulación: Debe basarse en el RankingActual de los equipos y utilizar una semilla de simulación (SemillaSimulation) para que el resultado sea reproducible en las pruebas de corrección.

2.7. Generación de fixture de primera fase del torneo

Condición previa: No se puede generar el fixture si no se tienen los 48 equipos cargados y al menos 4 estadios.

Se deben generar 12 grupos (A–L) de 4 equipos y 6 partidos por grupo, fechas y estadios por rotación.

Para realizar el fixture, los equipos deben estar ordenados en 4 bombos de 12 equipos según el orden:

- Orden principal: RankingFIFA DESC.
- Empates: sorteo determinístico (Fisher–Yates) con SemillaFixture (obligatoria).

Distribución en grupos (round-robin + confederaciones):

- Asignación A→L en round-robin sobre el orden anterior.

- UEFA puede repetir hasta 2 equipos en un grupo, el resto de confederaciones no.

Calendario por grupo:

- 6 partidos (ida única): Jornada 1 (E1 vs E4, E2 vs E3); Jornada 2 (E1 vs E3, E2 vs E4); Jornada 3 (E1 vs E2, E3 vs E4).
- Fechas: desde Fecha de inicio del torneo (default 2026-06-01), separación de 3 días (D, D+3, D+6).
- Si hay estadios disponibles, utilizarlos, pero el tope máximo es de 3 partidos por día con una separación de 4hs entre cada partido.
- La hora de inicio de cada fecha es a las 14:00hs.
- La última fecha de cada grupo se debe jugar el mismo día a la misma hora.

Rotación de estadios:

- Ordenar estadios por nombre normalizado asc.
- Asignación circular: Partido #n $\rightarrow$ Estadio[(n-1) mod CantEstadios].

2.8. Generar cruces por sorteo para segunda fase del torneo

Al finalizar la fase de grupos, se determina el orden de los equipos aplicando los siguientes criterios en este orden:

- Puntos (Pts): total de puntos obtenidos.
- Diferencia de goles (DG): goles a favor menos goles en contra.
- Goles a favor (GF): total de goles convertidos.
- Desempate final por sorteo determinístico: si persiste el empate tras los criterios anteriores, se aplica un sorteo reproducible usando una semilla (SemillaCrucesFase).

Con este ranking:

Se ordenan los 12 primeros y segundos de cada grupo y se seleccionan los 8 mejores terceros para completar los 32 equipos.

- Los 8 primeros con mayor puntaje se cruzan con los 8 terceros (de aquí se generan los partidos: A1, A2, A3, A4, A5, A6, A7, A8).
- Los 4 primeros restantes se cruzan con los 4 segundos con menor puntaje (de aquí se generan los partidos: B1, B2, B3, B4).
- Los 8 segundos restantes se cruzan entre sí (de aquí se generan los partidos: B5, B6, B7, B8).

Para realizar los cruces, se barajan ambas listas con un algoritmo determinístico (Fisher-Yates) usando la semilla SemillaCrucesFase. Cada equipo se intenta emparejar con el primer equipo disponible que no pertenezca al mismo grupo. Si no es posible, se pasa al siguiente.

Octavos de final

- Partidos: C1 = Ganador de A1 vs Ganador de B1
- Partidos: C2 = Ganador de A2 vs Ganador de B2
- Partidos: C3 = Ganador de A3 vs Ganador de B3
-
- Partidos: C8 = Ganador de A8 vs Ganador de B8

Cuartos de final

- Partidos: D1 = Ganador de C1 vs Ganador de C2
- Partidos: D2 = Ganador de C3 vs Ganador de C4
- Partidos: D3 = Ganador de C5 vs Ganador de C6
- Partidos: D4 = Ganador de C7 vs Ganador de C8

Semifinal

- Partido: S1 = Ganador de D1 vs Ganador de D2
- Partido: S2 = Ganador de D3 vs Ganador de D4

Tercer puesto

- Perdedor de S1 vs Perdedor de S2

Final

- Ganador de S1 vs Ganador de S2

2.9. Importación

Importar CSV (48 equipos) con encabezados exactos: Nombre, Confederación, RankingFIFA.

Validar nombre único, confederación válida y rango;

2.10. Auditoría (log)

Registrar con timestamp ISO-8601 y usuario:

- Altas/ediciones de usuarios.
- Altas, ediciones y eliminaciones de estadios
- Altas, ediciones y eliminaciones de equipos
- Generación automática de equipos
- Generación de fixture
- Modificación de partidos

- Realización de sorteo para cruces
- Importación de equipos, casos de éxito y errores

2.11. Pantallas mínimas (UI)

1. Login.
2. Usuarios (ABM) — solo para Administrador.
3. Equipos: Con generador automático para completar los 48 requeridos.
4. Estadios — solo para Administrador.
5. Fixture y cruces: Se debe poder generar el fixture y mostrar los 12 grupos con sus participantes. Al acceder al grupo se debe mostrar los equipos participantes, el estadio y fecha en que se jugarán los partidos. A su vez, se debe permitir generar los cruces para la segunda fase una vez completada la primera.
A medida que avanza el torneo se deben ir mostrando los ganadores y los cruces para ver visualmente cómo se desarrolla el mundial.
6. Listado de partidos: se deben mostrar organizados por fecha, se deben poder filtrar por fecha, estadio, grupo y fase.
7. Edición de partido: Se puede modificar la fecha, estadio o cargar el resultado del partido.
8. Visualización de log.
9. Importar.

3. Requerimientos asociados al desarrollo con IA Generativa

Como parte de este curso, se espera que los estudiantes comiencen a utilizar y evaluar el uso de agentes de IA que los asistan en el desarrollo de la aplicación. A continuación, se detallan los usos esperados de este tipo de herramientas.

3.1. Implementación de un requerimiento funcional

Se espera que el requerimiento funcional Gestión de Usuarios (ABM) sea desarrollado utilizando IA. La IA puede emplearse para generar total o parcialmente la implementación del requerimiento. La implementación debe incluir los tests unitarios asociados, cumplir con las prácticas de Clean Code, al igual que el código desarrollado por los programadores. La implementación debe seguir la estructura y convenciones que se utilizan para las funcionalidades implementada sin IA.

3.2. Utilización de Skills e instrucciones para el agente de IA

Una de las ventajas de este tipo de herramientas es la posibilidad de crear o reutilizar skills para que el agente realice tareas repetitivas (<https://docs.github.com/en/copilot/how-tos/use-copilot-agents/coding-agent/create-skills>). Asimismo, se pueden definir instrucciones generales a nivel de repositorio para que el agente las tenga en cuenta al momento de generar

código (<https://docs.github.com/en/copilot/how-tos/configure-custom-instructions/add-repository-instructions>).

Se solicita utilizar al menos una de las siguientes skills, disponibles en: <https://awesome-copilot.github.com/skills/>

Lista de skills recomendadas:

- Add Educational Comments: <https://awesome-copilot.github.com/skills/#file=skills%2Fadd-educational-comments%2FSKILL.md>
- Create-readme: <https://awesome-copilot.github.com/skills/#file=skills%2Fcreate-readme%2FSKILL.md>
- csharp-mstest: <https://awesome-copilot.github.com/skills/#file=skills%2Fcsharp-mstest%2FSKILL.md> (aplicable solo para código generado por IA)
- git-commit: <https://awesome-copilot.github.com/skills/#file=skills%2Fgit-commit%2FSKILL.md>

De forma opcional, se puede crear una nueva skill que asista al equipo en alguna tarea que lo justifique.

Importante: algunos skills pueden modificar el código de forma no esperada. Se recomienda probarlas en un proyecto de prueba antes de utilizarlas en el obligatorio.

3.3. Generación de alternativas de implementación

Para el requerimiento de Importación, se solicita generar, utilizando un agente, tres implementaciones distintas del algoritmo. Cada implementación debe priorizar un criterio diferente entre los siguientes:

- Extensibilidad: esta implementación debe facilitar la adaptación a cambios, por ejemplo, permitir agregar un nuevo campo al archivo de datos.
- Bajo acoplamiento y alta cohesión: esta implementación debe estar lo menos acoplada posible al resto de las clases de la solución.
- Facilidad de comprensión: esta implementación debe hacer énfasis en cumplir aquellas prácticas de clean code relacionadas a nombres, parámetros, uso de constantes y cumplimiento de métodos que hagan una sola cosa.

Las implementaciones deben desarrollarse en ramas del repositorio dedicadas a cada una y estar debidamente identificadas.

El equipo debe seleccionar una de las implementaciones de forma justificada y utilizarla en su solución.

Definiciones importantes:

- **Normalización:** minúsculas, sin tildes, no alfanuméricos → espacio, colapsar espacios, trim.
- **Determinismo con semillas:** Sorteos realizados mediante `System.Random(Semilla)` para garantizar que, ante una misma entrada, el resultado sea idéntico. Cada uso de una semilla debe quedar registrado en el log de auditoría (con timestamp y usuario) para asegurar la trazabilidad y reproducibilidad completa del sistema.
- **Algoritmo de Fisher-Yates:** Técnica de barajado determinístico utilizada para asegurar que todas las permutaciones de una lista sean igualmente probables. En este proyecto, es el método obligatorio para resolver empates de RankingFIFA, ordenar equipos en bombos y barajar las listas de clasificados en los cruces de eliminación.
- **Round-robin de grupos:** Distribución A→L sobre la lista de equipos previamente ordenada (donde los empates de ranking se resolvieron mediante Fisher-Yates).
- **Rotación de estadios:** Asignación circular de partidos a estadios siguiendo el orden de sus nombres normalizados ascendentemente.

Parámetros configurables para pruebas

Para garantizar la reproducibilidad y facilitar la verificación en las correcciones, los siguientes parámetros deben ser configurables (por UI o archivo de configuración):

- **SemillaFixture:** Semilla utilizada por el algoritmo Fisher-Yates para el sorteo determinístico en la generación del fixture, incluyendo el orden de equipos en bombos y la resolución de empates en el RankingFIFA.
- **SemillaCompletar:** Semilla para la generación automática de equipos ficticios (ej. UEFA_01) con un RankingFIFA reproducible cuando no se alcanzan los 48 equipos.
- **SemillaCrucesFase:** Semilla aplicada al algoritmo Fisher-Yates para barajar las listas de equipos clasificados y generar los emparejamientos de la segunda fase (Round of 32) y subsiguientes.
- **SemillaSimulation:** Parámetro esencial para que los resultados de los partidos simulados (basados en RankingActual) sean consistentes y verificables durante las pruebas.
- **FechaInicioTorneo:** Fecha base para el calendario (por defecto 2026-06-01).
- **MaxPartidosPorDia:** Tope máximo de partidos por jornada (por defecto 3).
- **SeparaciónEntreFechas:** Intervalo en días entre jornadas dentro de un grupo (por defecto 3).

Estos parámetros permiten reproducir resultados en diferentes entornos, evidenciar el cumplimiento de los algoritmos determinísticos exigidos y simplificar las pruebas automatizadas (TDD) al esperar resultados predecibles tras cada sorteo.

Nota sobre requerimientos

Nota: La totalidad y detalle de los requisitos serán relevados a partir de consultas en el [foro](#) correspondiente en aulas, las que deben tener un título descriptivo de su contenido. Tener en cuenta al momento de planificar el trabajo que las respuestas en Aulas pueden demorar hasta 48 horas y dicho foro de consultas cerrara el día 08/05/2026 23:59hs Para evitar complejidades innecesarias se realizaron simplificaciones al dominio del problema real.

A continuación, se describen los requisitos esperados para la entrega del obligatorio:

Diseño e Implementación:

Se debe entregar una aplicación que contemple toda la funcionalidad descrita en este documento. Para esta primera instancia la solución guardará toda la información en memoria, es decir, no es necesario el uso de base de datos (esto se implementará en la segunda entrega).

El desarrollo de todo el obligatorio debe cumplir:

- ≠ Estar en un repositorio Git, siguiendo Gitflow.
- ≠ El nombre del repositorio debe estar conformado de la siguiente forma:
 - NumeroEstudiante1_NumeroEstudiante2_NumeroEstudiante3
- ≠ Haber sido desarrollado utilizando TDD (desarrollo guiado por pruebas) lo que involucra otras dos prácticas: escribir las pruebas primero (Test First Development) y refactoring. De esta forma se utilizan las pruebas unitarias para dirigir el diseño.
- ≠ Cumplir los lineamientos de Clean Code (capítulos 1 al 10, y el 12), utilizando las técnicas y metodologías ágiles presentadas para crear código limpio.
- ≠ Utilizar las convenciones de codificación (“idioms”) de C#: <https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>
- ≠ Se requiere escribir los casos de prueba automatizados con el framework de unit tests visto en el curso, documentando y justificando las pruebas realizadas (ver documentación).
- ≠ Como parte de la evaluación se revisará el nivel de cobertura de los test sobre el código entregado. Por lo que se debe entregar un reporte y un análisis de la cobertura de las pruebas justificando los valores obtenidos según el diseño elaborado para la solución.

Correcto uso de IA Generativa

Los alumnos serán plenamente responsables por todo el contenido generado mediante IA. En consecuencia, deberán demostrar comprensión del cómo, el porqué y el para qué del código y/o diseño obtenido.

El desconocimiento total o parcial del código o de las decisiones de diseño por parte de cualquier integrante del equipo será motivo de penalización en la evaluación.

Documentación:

La documentación debe entregarse en un único documento digital (máximo 15 páginas, sin contar anexos) en formato PDF. El contenido debe estar organizado, indexado y estructurado según se indica a continuación:

Carátula del documento:

- Debe elaborarse según lo indicado en el documento 302.
- Debe incluir el enlace al repositorio de la entrega.

Descripción general del trabajo y del sistema:

- Describir brevemente, en sus propias palabras, el objetivo del sistema (qué problema resuelve y para quién lo resuelve).
- Indicar si alguna funcionalidad no fue implementada o si existen errores conocidos (bugs). Estos deben describirse en esta sección.

Diagramas de paquetes:

El objetivo de estos diagramas es visualizar la estructura de namespaces (paquetes) del sistema y las dependencias entre ellos.

Se debe incluir:

- Diagramas de paquetes que muestren la organización de los namespaces definidos.
- No incluir los paquetes correspondientes al código de pruebas.
- Incluir paquetes y subpaquetes.
- No mostrar las clases dentro de los paquetes.
- Un diagrama de alto nivel que muestre la composición de paquetes y subpaquetes, así como sus dependencias a nivel de paquetes principales.
- Al menos un diagrama que muestre las dependencias entre paquetes hoja (aquellos que no contienen otros paquetes). Para identificarlos claramente, utilizar la notación UML de clasificadores (por ejemplo: paqueteA::paqueteB).

- Un breve análisis sobre si las dependencias favorecen la modificabilidad. En particular, describir el motivo de las dependencias desde la interfaz de usuario (front-end) hacia los paquetes de servicios y dominio (back-end).
- Una tabla que describa las responsabilidades de cada paquete, permitiendo comprender qué clases y subpaquetes contiene.

Ejemplo de tabla:

Paquetes	Responsabilidad
Dominio	
Dominio.Excepciones	

Diagramas de clases:

Estos diagramas permiten entender cómo colaboran los objetos para implementar la funcionalidad del sistema. Se debe prestar atención a la notación (tipo de relaciones, navegabilidad, multiplicidad, etc.). Los diagramas deben ser completos y formales.

Dado que el objetivo es comunicar el comportamiento de las clases, es suficiente mostrar properties y métodos.

Se deben incluir los siguientes diagramas, cada uno acompañado de una breve explicación:

- Diagrama de clases del dominio de la aplicación.
- Diagramas de clases a nivel de servicio, mostrando la relación entre servicios, clases de dominio y acceso a datos. Si existen relaciones entre servicios, también deben incluirse para comprender su colaboración.
- No incluir diagramas de la interfaz de usuario, salvo que implementen lógica de negocio.

Tabla de responsabilidades:

Se debe incluir una tabla que describa, para cada clase del dominio y de los servicios, sus responsabilidades (validaciones, procesamiento, etc.) y una breve justificación.

Ejemplo:

Clase	Responsabilidades	Justificación
Usuario		
ServicioABC		

Explicación de los mecanismos generales:

Describir los mecanismos generales del sistema. Por ejemplo:

- Interacción entre la interfaz de usuario y el dominio.
- Almacenamiento de datos.
- Manejo de errores y excepciones.
- Uso de polimorfismo.

Análisis detallado de dependencias:

Para el requerimiento “Generación de fixture”, identificar:

- Las clases del dominio involucradas.
- Las dependencias entre ellas.
- Un análisis de cohesión y acoplamiento.

Este análisis debe basarse en un diagrama de clases que incluya todas las clases involucradas (sin incluir clases de la interfaz de usuario, salvo que contengan lógica de negocio).

Aplicación de TDD (a nivel del repositorio):

La solución debe desarrollarse utilizando TDD. Se debe evidenciar el ciclo RED, GREEN y REFACTOR para el requerimiento “Generación de fixture”, de forma que permita comprender el proceso seguido.

Esta sección debe incluir:

- Un commit por cada test realizado.
- Indicación del paso del ciclo (RED, GREEN o REFACTOR) correspondiente a cada commit.

Para el resto de la funcionalidad, se recomienda realizar commits cuando los tests estén en estado GREEN.

Cobertura de pruebas unitarias:

Se debe incluir el reporte de cobertura generado con JetBrains dotCover, abierto a nivel de clase.

Si alguna funcionalidad no alcanza el 90% de cobertura de líneas, se debe incluir un análisis explicando por qué y qué aspectos no fueron cubiertos por los tests.

Pruebas funcionales:

Para las funcionalidades de Gestión de Usuarios (ABM), se deben documentar - en un Anexo - los casos de prueba funcionales, incluyendo:

- Valores inválidos
- Valores límite
- Tipos de datos incorrectos
- Datos vacíos o nulos
- Omisión de campos obligatorios
- Formatos inválidos
- Reglas de negocio

Se debe entregar evidencia de la ejecución de estas pruebas (no unitarias), mediante:

- Capturas de pantalla (en anexos), o
- Videos publicados (por ejemplo, en YouTube)

Los enlaces a los videos deben incluirse en el documento y verificarse para asegurar que sean accesibles por terceros.

Explicación del algoritmo:

Para el requerimiento “Generación de fixture”, se debe incluir en el anexo una descripción del algoritmo implementado, explicando su funcionamiento.

Dependencias externas:

Si se utiliza alguna dependencia externa (por ejemplo, para gráficos), se debe documentar en el anexo:

- Nombre de la dependencia
- Versión utilizada

Uso de IA Generativa

Para aquellas funcionalidades implementadas con apoyo de IA generativa, se pide:

Implementación de un requerimiento funcional

- Un análisis crítico de la calidad del código generado, fundamentado en los principios teóricos vistos en el curso (por ejemplo: Clean Code, cohesión y acoplamiento, y calidad de las pruebas unitarias).

Utilización de Skills e instrucciones para el agente de IA

- Documentar los skills o archivos de instrucciones utilizados y asegurarse que se encuentren en el repositorio.
- demostrar en base a algún ejemplo el resultado del uso de los skills o archivos de instrucciones.

Generación de alternativas de implementación

- Para cada implementación identificar el nombre de la rama que la contiene.
- Identificar la implementación seleccionada y explicar brevemente el motivo por el cual la seleccionaron.

Sobre los anexos:

Los anexos deben utilizarse únicamente para profundizar en aspectos específicos. El lector debe poder comprender el contenido principal sin necesidad de consultarlos.

Importante:

Se espera que el documento siga un orden lógico, integrando diagramas y explicaciones según corresponda. La entrega será evaluada como si estuviera dirigida a un cliente real, considerando claridad, prolijidad y profesionalismo.

Entrega por Gestión

Es de carácter obligatorio que la última versión de la documentación sea entregada vía Gestión. En caso de que se quieran adjuntar los diagramas que contiene el documento sueltos como imágenes se deben subir por Gestión en archivo zip.

No se aceptará documentación que se incluya en el repositorio en github.

Entrega en el repositorio de GitHub

La entrega en el repositorio del equipo en la organización de GitHub ([IngSoft-DA1](#)) debe estar compuesta por los siguientes elementos:

- € Una carpeta con la aplicación compilada.
- € Código fuente de la aplicación y de los tests (merge a main), incluyendo el proyecto que permita probar y ejecutar.
- € Release y Tag en la rama main con un nombre adecuado (ver <https://docs.github.com/es/repositories/releasing-projects-on-github/managing-releases-in-a-repository>). La hora tope para realizar el merge a main y crear el Release / Tag será - sin excepción - las 21:00 del día fijado para la entrega.
- € La documentación NO DEBE incluirse en el repositorio, la misma solo se puede entregar por [Gestión](#).

Rúbrica de evaluación

Puntaje total: 20 puntos.

	Evaluación de	Pts.	Criterios de corrección
Funcionalidad	Implementación de la funcionalidad pedida y calidad de la interfaz de usuario.	6	Excelente: Se implementa toda la funcionalidad, tiene buena usabilidad y no hay errores importantes de funcionamiento. Correcto: Falta algún punto no central de la funcionalidad, existen detalles no bloqueantes de funcionamiento o la aplicación no es fácil de usar. No suficiente: Faltan partes importantes (por alcance o dificultad) de la funcionalidad. Existe gran cantidad de errores o funcionalidades no usables.
Diseño y documentación	Cumplimiento con los ítemas solicitados en el apartado documentación	5	Excelente: Se presenta un documento completo en función de lo que se pide, ordenado y prolijo, que explica la solución entregada en todos sus aspectos. Se utiliza la notación vista en clase, de manera formal y correcta, para presentar los aspectos importantes del diseño. Se intercalan diagramas y explicaciones, que permiten comprender el diseño de la solución, las decisiones tomadas y la motivación detrás de las mismas. Correcto: Se presenta un documento prolijo que

			describe el diseño de la solución. Se describen los aspectos más importantes, pero hay errores en el uso de la notación. Falta describir aspectos importantes solicitados El orden de la documentación no permite seguir un hilo descriptivo. No suficiente: Falta detallar parte importante de la solución. No se usa la notación vista en clase, o se usa en forma equivocada o incompleta en repetidas ocasiones. Se presenta la documentación como una sucesión de diagramas sin explicación, o se omiten completamente items solicitados.
Calidad del código y metodología de desarrollo	Uso de prácticas de control de versiones y configuración. Desarrollo guiado por las pruebas (TDD) y técnicas de refactorio de código Buenas prácticas de estilo y codificación y su impacto en la mantenibilidad (Clean Code) Correcto uso de las tecnologías Concordancia entre el código y el diseño documentado	6	Excelente: El código es prolijo, fácil de entender y cumple con los puntos cubiertos en el curso sobre Clean Code. Lo construido coincide con lo detallado en la documentación de diseño. Se evidencia el uso de TDD mediante los commits en el repositorio. Hay buena cobertura de casos de prueba. Correcto: El código es prolijo en general, aunque presenta detalles a mejorar. Lo construido se corresponde en términos generales con lo presentado en el documento. Hay pruebas unitarias, aunque no se evidencie el uso de TDD. No suficiente: El código es desprolijo o difícil de entender. No cumple en repetidos casos lo propuesto por Clean Code. No hay pruebas unitarias.
Requerimientos de IA	Uso de la IA generativa para los	3	Excelente: Se evidencia un uso sistemático de la IA

Generativa	requerimientos solicitados (código, skills, y alternativas de implementación). Completitud de los requerimientos asociados al uso de IA solicitados. Documentación completa y correcta respecto a lo solicitado	generativa en el requerimiento solicitado. La implementación concuerda con el diseño definido para el código no generado. Se utiliza <i>skills</i> o instrucciones bien definidas. Se documenta el resultado del uso de la IA para generar alternativas y compararon alternativas de solución. La documentación es clara, precisa y permite comprender el uso de las herramientas. Correcto: Se utiliza IA generativa en forma parcial para implementar funcionalidades, el uso de skills o instrucciones podría ser más claro o sistemático. Se evidencia cierto uso del agente para implementar alternativas de implementación, pero de forma limitada o poco profunda. La mayoría de los requerimientos solicitados están implementados, aunque puede haber omisiones menores. La documentación describe el uso de IA, pero carece de detalle que permita al lector comprender bien los items solicitados. No suficiente: El uso de IA generativa es superficial, poco claro o inexistente en alguno de los aspectos solicitados. No se evidencia el uso de <i>skills</i> o instrucciones relevantes, ni el análisis de alternativas con apoyo del agente. Faltan requerimientos asociados al uso de IA o están incompletos. La documentación es insuficiente, confusa o no permite entender cómo se utilizó la IA ni qué decisiones se tomaron.
------------	--	--

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

- **Obligatorios**

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

Cualquier integrante podrá:

- **Modificar la integración del equipo.**
- **Subir el archivo de la entrega.**

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 3 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. **La hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador de cursos o Coordinación adjunta antes de las 20:00h del día de la entrega, a través del correo adjuntos_ei@ort.edu.uy, o vía Ms Teams de la Bedelía.